

Phase 5 - limitations

- [PHASE 5 — LIMITATIONS](#)
 - [1. ai.documents.embedding is intentionally left empty](#)
 - [2. LSA is not a transformer embedding — said plainly, everywhere it matters](#)
 - [3. Chunking is a documented no-op on the current dataset](#)
 - [4. Small corpus \(120 documents\)](#)
 - [5. Grounding validates numbers, not meaning](#)
 - [6. Prompt-injection detection is pattern-based](#)
 - [7. RAG evaluation measures retrieval, not full answer quality](#)
 - [8. No live PostgreSQL was available to run the DB-default path in this environment](#)

PHASE 5 — LIMITATIONS

Consolidated from the phase report. Kept as a separate document per Master Prompt Section 16 (Documentation must cover “Limitations” as a first-class topic).

1. `ai.documents.embedding` is intentionally left empty

The Phase 1 schema reserves a `vector(384)` column for a transformer sentence-embedding model. No such model (e.g. a sentence-transformers checkpoint) was available to download in this offline environment. Rather than fill that column with something that isn’t actually a transformer embedding — which would be misleading to any future consumer of `ai.documents` — this phase’s semantic retrieval signal (LSA, via `TruncatedSVD` on a TF-IDF matrix, ≤ 40 components) is computed and used at query time, but never written into the `embedding` column. A future phase can populate real embeddings there without needing any change to this phase’s retrieval interface (`HybridRetriever` would simply gain a third scoring signal).

2. LSA is not a transformer embedding — said plainly, everywhere it matters

Per Master Prompt Section 5.5’s explicit instruction, every place `semantic_retrieval.py` is used or described states that LSA is a classical linear-algebra, corpus-statistics-based signal (captures term co-occurrence “topics”), not a neural/contextual embedding. Its failure modes are different: LSA cannot understand a paraphrase that shares no vocabulary at all with the corpus, in the way a transformer embedding sometimes can. This is a real capability boundary, not a minor wording issue.

3. Chunking is a documented no-op on the current dataset

`ai.documents.text` averages 221 characters (min 211, max 232) across all 120 rows — well under the 800-character `CHUNK_SIZE_CHARS` default, so every document currently becomes exactly one chunk. The sentence-aware, overlap-carrying chunker in `chunking.py` is fully implemented and unit-tested (both in `test_phase5.py` and in the module’s own `__main__` block) against a synthetic 4,369-character document, where it correctly produces 20 overlapping chunks. It has not yet been exercised against real long-form content (e.g. a full PDF market report) because the Phase 1 dataset does not currently contain any documents that long.

4. Small corpus (120 documents)

BM25 and LSA both function correctly and were verified against real queries at this scale, but no claim is

made about retrieval-quality-at-scale (thousands or millions of documents). Index-build time, candidate-pool size (`candidate_pool=40` in `hybrid_retrieval.py`), and reranking behavior would all need re-tuning and re-measurement if the corpus grows substantially.

5. Grounding validates numbers, not meaning

`grounding.py` 's claim validation checks that a cited number actually appears (within 1% tolerance) in the cited evidence's text. It does NOT verify that the number is being used with the correct *meaning* — e.g., citing a real “connectivity score of 59.9” from a document, but describing it as a “safety score,” would currently pass the number-matching check even though the claim is wrong. This is a deliberate, documented scope boundary (full semantic fact-checking is a much harder, LLM-dependent problem, and the Master Prompt explicitly wants deterministic, non-LLM evidence gates for this layer — Section 3.4).

6. Prompt-injection detection is pattern-based

`prompt_injection.py` uses 11 documented regex patterns covering instruction override, role override, system-prompt probing, authority spoofing, role-tag injection, and concealment instructions. It correctly caught all 3 malicious examples and passed both clean examples in its adversarial test set, and found zero false positives across the real 120-document corpus. It will NOT catch a novel injection phrasing outside these patterns — this is a first line of defense, not a guarantee, and is documented as such directly in the module so a future maintainer does not over-trust it.

7. RAG evaluation measures retrieval, not full answer quality

Phase 5 has no LLM call in it (that begins in Phase 9's Explanation Agent and Phase 10's AI Explanation Layer). `rag_evaluation.py` 's six test categories (direct, paraphrased, ambiguous, no-answer, conflicting evidence, unsupported) therefore evaluate whether the *retrieval* layer surfaces (or correctly fails to surface) the right evidence — not whether a downstream LLM would write a good final answer from that evidence. This is a legitimate, narrower evaluation, stated explicitly here rather than implied to be more than it is.

8. No live PostgreSQL was available to run the DB-default path in this environment

This sandbox has no PostgreSQL instance and no network access to install one, and `psycopg2` is not installed. Every orchestrator script (`generate_evidence_report.py` , `export_results_csv.py` , `generate_charts.py`) defaults to `--source db` — the production path, matching Phase 3's `db/connection.py` and its “No CSV dependency” convention exactly — but was actually run and re-checked here with `--source csv` , a documented dev/test-only path that reads the identical rows from `data/processed/documents.csv` (the same file originally loaded into `ai.documents`). Recommendation: re-run all three orchestrators with `--source db` (default) in the user's local environment where Phase 1/3's PostgreSQL database is already running, to confirm the DB path itself (not just its logic) end-to-end.